

Materia: Inteligencia Artificial**Fecha:** 18 de Diciembre, 2024**Autor:** Benítez Gómez Josué Miguel

DOCUMENTACIÓN DE PRÁCTICA

Implementación de Fine-Tuning de LLM en Entorno Local
(Windows/AMD)

1. Objetivo General

El objetivo de la práctica fue implementar un proceso de ajuste fino (Fine-Tuning) supervisado sobre un Modelo de Lenguaje Grande (LLM) utilizando un dataset personalizado (.jsonl) de tutoría de programación. El reto principal consistió en lograr la ejecución en un entorno local hostil para el aprendizaje profundo (Windows + GPU AMD), sin depender de herramientas externas de abstracción como Ollama, utilizando código Python puro y el ecosistema de Hugging Face.

2. Entorno de Desarrollo y Hardware

El despliegue presentó desafíos significativos debido a la incompatibilidad nativa de las librerías estándar con la configuración de hardware disponible.

- **Sistema Operativo:** Windows 11
- **Procesador (CPU):** Arquitectura x64
- **Tarjeta Gráfica (GPU):** AMD Radeon RX 6600
- **Librerías Clave:** transformers, peft (LoRA), trl (SFTTrainer), datasets.

3. Cronología de Problemas (Troubleshooting)

El desarrollo siguió un enfoque iterativo de resolución de errores, enfrentando múltiples capas de incompatibilidad.

Fase 1: El "Infierno de las Dependencias"

Al iniciar el entorno, surgieron conflictos entre las versiones modernas de Keras y la librería Transformers.

ValueError: Your currently installed version of Keras is Keras 3, but this is not yet supported in Transformers.

Solución: Se instaló el paquete puente `tf-keras` para mantener la compatibilidad legacy sin romper el entorno de TensorFlow existente.

Fase 2: Inestabilidad de la API TRL

La librería `trl` se actualizó recientemente, deprecando argumentos que la documentación oficial aún cita.

```
TypeError: SFTTrainer.__init__() got an unexpected keyword argument 'max_seq_length'  
TypeError: SFTTrainer.__init__() got an unexpected keyword argument 'tokenizer'
```

Solución: 1. Migración de argumentos a la nueva clase `SFTConfig`.
2. Cambio del parámetro `tokenizer` a `processing_class`.
3. Eliminación de restricciones de longitud explícitas para usar los valores por defecto del modelo.

Fase 3: Formateo de Datos

El entrenador fallaba al intentar procesar el dataset por lotes dentro de la función de formateo interna.

```
AttributeError: 'list' object has no attribute 'endswith'
```

Solución: Se implementó un pre-procesamiento manual con `dataset.map()`, formateando el texto antes de entregarlo al entrenador y eliminando la lógica ambigua del `SFTTrainer`.

Fase 4: Bloqueo de Hardware (El reto de AMD en Windows)

El entrenamiento se congelaba indefinidamente ("Deadlock") o no iniciaba.

```
UserWarning: 'pin_memory' argument is set as true but no accelerator is found
```

Diagnóstico: PyTorch en Windows no soporta aceleración nativa para tarjetas AMD (ROCm). El intento de multiprocesamiento saturaba la CPU.

Solución Final:

1. Forzar modo CPU: `use_cpu=True`
2. Desactivar multiprocesamiento: `dataloader_num_workers=0`
3. Usar precisión simple: `fp16=False` (La CPU no maneja bien la media precisión).

4. Implementación Exitosa

Se substituyó el modelo inicial *TinyLlama* por **Qwen 2.5-0.5B-Instruct** debido a su mejor arquitectura y eficiencia. A continuación, el fragmento clave del código final:

```
# Configuración adaptada para Windows/CPU
training_args = SFTConfig(
    output_dir="./resultados_qwen",
    num_train_epochs=3,
    per_device_train_batch_size=1, # Lote mínimo para no saturar RAM
    gradient_accumulation_steps=4,
    learning_rate=1e-4, # --- AJUSTES CRÍTICOS ---
    bf16=False, fp16=False, # Precisión completa (Float32)
    use_cpu=True, # Evita búsqueda de CUDA
    dataloader_pin_memory=False,
    dataloader_num_workers=0, # Evita bloqueos en Windows
    optim="adamw_torch" # Optimizador compatible con CPU
)
```

5. Resultados

Métrica	Valor Obtenido
Modelo Base	Qwen/Qwen2.5-0.5B-Instruct
Tiempo de Entrenamiento	~11 minutos (3 épocas)
Training Loss Final	1.449
Estado	Convergencia Exitosa

El valor de pérdida (Loss) de 1.44 indica que el modelo logró aprender patrones del dataset de manera efectiva, a pesar de las limitaciones de hardware.

6. Conclusión

La práctica demostró que es posible realizar *Fine-Tuning* de modelos de lenguaje modernos en hardware de consumo y sin soporte oficial de CUDA (NVIDIA), aunque requiere una configuración manual exhaustiva. Se aprendió la importancia de gestionar versiones de librerías volátiles (como `tr1`) y de adaptar los pipelines de datos (Data Loaders) a las limitaciones del sistema operativo Windows cuando se trabaja con hardware AMD.

Reporte generado automáticamente tras la ejecución exitosa del script `trainer.py`.